

Coding for Artificial Intelligence

Module 1: Computational Foundations

Problem Set 1

Computational Foundations

Task: The Temporal Profile Analyzer

Course	Coding for AI (Module 1)
Author	Dr. Prateek, School of AI
Duration	One and a half hours
Late-Submission	One (1) week from the date of issue [Penalty will be added 50% of scored marks]
Total Marks	100 (+10 Bonus)

Read this document in full before you begin writing code.

Contents

1	Objective	2
2	Problem Statement	2
3	Functional Requirements	2
3.1	Name Analysis	2
3.2	Temporal Projection	2
3.3	The AI Readiness Formula	3
4	Technical Constraints (The “Standard” Requirements)	3
5	Edge Cases for Consideration	4
6	Deliverables	4
7	Challenge Task (Bonus, +10 Marks)	5
	Academic Integrity	5

1 Objective

The purpose of this problem set is to move beyond elementary scripting and to demonstrate command over four foundational pillars of Python programming:

- (i) Python’s core data types and dynamic typing model,
- (ii) explicit type conversion and its correct use,
- (iii) string processing and formatting, and
- (iv) the PEP 8 (Python Enhancement Proposal) style guide for readable, maintainable code.

You will implement a small interactive program, the **Temporal Profile Analyzer**, which accepts a user’s personal metadata and computes an “*AI Era Readiness Score*” using precise arithmetic and string manipulation. While the scenario is illustrative, the engineering discipline required to complete it correctly is not: careful validation, precise arithmetic, and clean style are exactly what is expected of production-grade code.

2 Problem Statement

Data is often described as the primary currency of the AI era. Your task is to build a robust, interactive command-line module that performs three operations:

1. **Ingests and tokenizes** a user’s full name to analyze its computational “weight” (i.e., its length in characters).
2. **Calculates temporal metrics**, based on the user’s current age and a projected age at a future reference year.
3. **Evaluates an AI Readiness Score** using a derived mathematical expression that combines the two quantities above.

3 Functional Requirements

Your program must implement the following logic precisely.

3.1 Name Analysis

1. Prompt the user for their full name using `input()`.
2. Compute the length of the string (after removing leading and trailing whitespace) and report it as the **Identifier Byte-Count**.
3. Convert the name to Title Case, so that the stored record is consistent regardless of how the user typed it (e.g., “mtech ai” → “Mtech Ai”).

3.2 Temporal Projection

1. Accept the user’s current age as a string from `input()`.
2. Validate the string using `str.isdigit()` before converting it, and only then cast it to an `int` using `int()`.
3. Using the current calendar year, calculate the user’s projected age in the year **2045** — a theoretical “Singularity” reference point used for this exercise only.

Note on integers in Python. Unlike languages such as C or Java, Python’s built-in `int` type is **not** a fixed-width (e.g., 32-bit or 64-bit) integer. It has arbitrary precision, limited only by available memory. When we say “cast to an integer,” we mean using `int(user_input)`; there is no notion of a “64-bit integer” to select in native Python, and you should not attempt to imitate one.

Note on obtaining the current year. Do not hard-code the current year as a literal. Use the `datetime` module from the standard library, specifically `datetime.date.today().year`, so that your program remains correct regardless of when it is run.

3.3 The AI Readiness Formula

Compute a score using the following expression:

$$\text{Score} = \frac{(\text{Name Length} \times 10) + \text{Age in 2045}}{2} \quad (1)$$

The result must be reported as a floating-point number rounded to exactly two decimal places (use Python’s formatted string literals, e.g., `f"{score:.2f}"`).

Worked Example. Suppose the user enters the name "Scooby-Dooo" (Name Length = 12, ignoring the leading/trailing whitespace but counting the internal space) and a current age of 20 in the year 2026. Then:

$$\text{Age in 2045} = 20 + (2045 - 2026) = 39$$

$$\text{Score} = \frac{(12 \times 10) + 39}{2} = \frac{159}{2} = 79.50$$

Your program, when run with these exact inputs in the year 2026, should produce the score 79.50.

4 Technical Constraints (The “Standard” Requirements)

To meet the standard expected of rigorous, top-tier computing courses, your submission will be evaluated against the following constraints.

1. **PEP 8 Compliance.** Use `lowercase_with_underscores` for all variable and function names (e.g., `user_full_name`, not `x` or `userFullName`).
2. **Memory Discipline.** Avoid creating redundant intermediate variables. Reuse names where it is logical and does not harm readability; do not, however, sacrifice clarity purely to save a line.
3. **Robust Input Handling.** Full exception-handling (`try/except`) is covered in a later module. For this assignment, you must instead perform *basic* validation using `str.isdigit()` before every numeric conversion. If validation fails, your program should print a clear, human-readable message and terminate gracefully, rather than crashing with an unhandled traceback.
4. **Documentation.** Every submitted file must begin with a top-level docstring (enclosed in triple quotes, `"""`) that states the program’s purpose, the author’s name, and the date.

```
1 """
2 Program : Temporal Profile Analyzer
3 Purpose : Computes an AI Era Readiness Score from user metadata.
4 Author  : <Your Full Name>
5 Date    : <Submission Date>
6 """
7
8 import datetime
9
10 user_full_name = input("Enter your full name: ").strip()
11 name_length = len(user_full_name)
12 formatted_name = user_full_name.title()
```

5 Edge Cases for Consideration

A program is only as strong as its handling of the inputs its author did not anticipate. Before submitting, trace through your code manually for each of the following scenarios and be prepared to explain your program's behaviour for all three:

1. A user enters a name consisting only of whitespace characters.
2. A user enters a non-numeric string (e.g., "twenty") when prompted for their age. Your validation step should catch this *before* attempting `int()` conversion, since an unguarded call would raise a `ValueError`.
3. A user enters an age of 0, or an age that, once projected to 2045, would be mathematically valid but practically unusual (e.g., a very large age). Consider whether your formula behaves sensibly at these boundaries, and state any assumptions you make about valid input ranges.

6 Deliverables

1. **Source Code:** A single file named `assignment1_readiness.py`.
2. **Console Log:** A screenshot (or pasted terminal output) showing:
 - (a) one "*Perfect Run*" with valid inputs, and
 - (b) one "*Failed Run*" in which invalid input is correctly detected and handled by your program.

All files must be submitted through the LMS-course portal by the deadline stated on the title page (Zipped all files together and tagged as **[RollNo-Assignment1-Name.zip]**). Late submissions will be handled according to the course's standard late-submission policy.

7 Challenge Task (Bonus, +10 Marks)

Implement a string-repetition routine that concatenates the user's (formatted) name with itself a number of times equal to the **first digit** of their age.

Worked Example. If the user's age is 22 and their name is "Ai", the first digit of 22 is 2 (obtained via `age // 10` for any two-digit age), so the output should be "AiAi".

Hint. For a two-digit age, integer division by 10 (`age // 10`) isolates the leading digit. Think carefully about what this expression would return for a single-digit age (e.g., 7), and handle that case explicitly so your program does not silently produce an empty or incorrect string.

Academic Integrity

You may discuss approaches with peers, but the code you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied, from a peer or from an external source, will be treated in accordance with the institute's academic integrity policy.